LAB SESSION 04

Doubly Linked List Operations

A doubly linked list (DLL) is a type of linked data structure where each node stores two pointers, one pointing to the previous node and the other pointing to the next node. This allows movement in both forward and backward directions, offering more flexibility than singly linked lists.

Structure of a Node:

```
struct Node {
int data;
Node* prev;
Node* next;
};
```

1. Data – the value stored in the node.

2. Next pointer – Address of the next node (NULL for the last node).

3. Previous pointer – Address of the previous node (NULL for the first node).

Key Operations

1. Traversal ((Forward and Backward)

2. Insertion

● At Beginning – Link the new node's next to the current head and set head's prev to the new node, then update head.

● At End – Link the new node's prev to the last node and last node's next to the new node.

● At Position – Adjust prev and next pointers of surrounding nodes to fit the new node in between.

3. Deletion

● Deletion at Beginning – Update head to the second node, set its prev to NULL, and free the removed node's memory.

● Deletion at End – Move to the last node, update the second-last node's next to NULL, and free

the last node.

● Deletion at a Specific Position – Adjust the next of the previous node and the prev of the next

Feature	Singly Linked List	Doubly Linked List
Memory per node	2 fields	3 fields
Backward traversal	No	Yes
Insertion/deletion	Requires extra steps	Easier (if previous pointer available)
Performance	Less flexible	More flexible

Advantages:

● Traversal possible in both directions.

● Faster insertion/deletion at both ends.

Disadvantages:

● Requires more memory per node.

● Slightly more complex pointer management.

1. ALGORITHM FOR TRAVERSING A DOUBLY LINKED LIST (Forward)

Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Steps 3 and 4 while PTR ≠ NULL
Step 3: Apply Process to PTR → DATA
Step 4: SET PTR = PTR → NEXT
[END OF LOOP]
Step 5: EXIT

Backward Traversal

Algorithm:

1. Start with a pointer ptr = tail.

2. Print "Backward: ".

3. Repeat until ptr == NULL: a. Print ptr → data followed by " <-> ". b. Move ptr = ptr → prev.

4. Print "NULL".

2. ALGORITHM: INSERT A NEW NODE AT THE BEGINNING OF THE DOUBLY LINKED LIST

Step 1: IF AVAIL = NULL, then
    Write OVERFLOW
    Go to Step 9
[END OF IF]

Step 2: SET New_Node = AVAIL
Step 3: SET AVAIL = AVAIL → NEXT
Step 4: SET New_Node → DATA = VAL
Step 5: SET New_Node → PREV = NULL
Step 6: SET New_Node → NEXT = START
Step 7: IF START ≠ NULL, then
    SET START → PREV = New_Node
[END OF IF]
Step 8: SET START = New_Node
Step 9: EXIT

3. ALGORITHM TO INSERT A NEW NODE AT THE END OF THE DOUBLY LINKED LIST

Step 1: IF AVAIL = NULL, then
    Write OVERFLOW
    Go to Step 12
[END OF IF]

Step 2: SET New_Node = AVAIL
Step 3: SET AVAIL = AVAIL → NEXT
Step 4: SET New_Node → DATA = VAL
Step 5: SET New_Node → NEXT = NULL

Step 6: IF START = NULL, then
    SET New_Node → PREV = NULL
    SET START = New_Node
    Go to Step 12
[END OF IF]

Step 7: SET PTR = START
Step 8: Repeat Step 9 while PTR → NEXT ≠ NULL
Step 9: SET PTR = PTR → NEXT
[END OF LOOP]

Step 10: SET PTR → NEXT = New_Node
Step 11: SET New_Node → PREV = PTR
Step 12: EXIT

TASKS:

1. Implement the above algorithms in C++.
2. Add a search function to find a given value.
3. Write a function to count the total number of nodes in the DLL.
4. Modify insertAtPosition() to handle inserting at the last position without using insertAtEnd().